



جامعة برج العرب التكنولوجية

Information Technology Department



جامعة برج العرب التكنولوجية
BORG AL ARAB TECHNOLOGICAL UNIVERSITY

كلية تكنولوجيا الصناعة والطاقة

BORG AL ARAB TECHNOLOGICAL UNIVERSITY

INTRODUCTION TO C PROGRAMMING ESSENTIALS

First Year –Information Technology Program

spring 2026

LECTURE 5

C LANGUAGE

BASICS

LOOPS

Dr. Dina Abdelhafiz & Dr. Ghada Fathy

LOOPS

- Loops can execute a block of code as long as a specified condition is true.]
- Loops are used and important because they save time, reduce errors, and they make code more readable.

WHILE LOOP

- The while loop repeats a block of code as long as a specified condition is true

Note: A while loop may never run if the condition is false from the start.

- Example:

```
#include <stdio.h>
int main()
{
    int i = 0;
    while (i < 5) {
        printf("%d\n", i);
        i++;
    }
    return 0;
}
```

Diagram illustrating the execution of the while loop:

- The **Condition** is $i < 5$.
- The **Loop body** contains the statements `printf("%d\n", i);` and `i++;`.
- The loop runs for $i = 0, 1, 2, 3, 4$, as indicated by the arrows and the output.
- The loop terminates when i reaches 5, as the condition $i < 5$ becomes false.

Output

0
1
2
3
4

WHAT'S WRONG HERE!!!

```
#include <stdio.h>

int main()
{
    int i = 0;
    while (i < 5) {
        printf("%d\n", i);
        i++
    }

    return 0;
}
```

$$\begin{array}{l} 5 \\ 0 \end{array} < 5$$

Output

0

0

0

0

0

0

0

0

0

0

15

0

2

Q

0

0

- Note:** Do not forget to increase the variable used in the condition (`i++`), otherwise the loop will never end!

COUNTDOWN EXAMPLE

- You can also use a while loop to count down. This example counts from 10 to 1, and then prints "Happy New Semester!!" at the end:

```
#include <stdio.h>
```

```
int main() {  
    int countdown = 10;  
  
    while (countdown > 0) {  
        printf("%d\n", countdown);  
        countdown--;  
    }  
    printf("Happy New Semester!!\n");  
    return 0;  
}
```

Condition is True

Output

10
9
8
7
6
5
4
3
2
1

Happy New Semester!!

CHOOSE THE CORRECT ANSWER

What is the purpose of the while loop in C?

- A. To execute a block of code a fixed number of times
- B. To execute a block of code as long as a specified condition is true
- C. To execute code only if a condition is false
- D. To declare multiple variables in a loop

What is the correct syntax for a while loop?

- A. `while { condition }`
- B. `while (condition) { // code block }`
- C. `while (condition) // code block`
- D. `while condition { // code block }`

Print i as long as i is less than 6.

```
int i = 1;
while(i < 6) {
    printf("%d\n", i);
    i++;
}
```

What will the following code output?

```
int i = 0;
while (i < 3) {
    printf("%d ", i);
    i++;
}
```

- A. 0 1 2
- B. 1 2 3
- C. 0 1 2 3
- D. No output

What happens if the condition in a while loop is always true and there is no increment statement?

- A. The loop runs indefinitely
- B. The loop runs once →
- C. The loop produces an error →
- D. The loop runs twice →

DO/WHILE LOOP

- The do/while loop will **execute the code block once**, **before** checking if the **condition is true**, then it will repeat the loop as long as the condition is true

```
#include <stdio.h>
```

```
int main() {  
    int i = 0;
```

```
    do {  
        printf("%d\n", i);  
        i++;  
    }
```

```
    while (i < 5);
```

```
    return 0;  
}
```

Output

0

1

2

3

4

condition

DO/WHILE LOOP

- do/while loop **will always be executed at least once**, even if the condition is **false**, because the code block is executed **before** the condition is tested
- Do not forget to increase the variable used in the condition, otherwise the loop **will never end** (The loop runs indefinitely)!

```
#include <stdio.h>
```

```
int main() {  
    int number;  
  
    do {  
        printf("Enter a positive number: ");  
        scanf("%d", &number);  
    } while (number > 0);  
  
    return 0;  
}
```

Handwritten notes:
- A blue line connects the `while (number > 0);` condition to the `while` keyword.
- A blue arrow points from the `while` keyword to the `do` block.
- A blue arrow points from the `while` block to the `while` keyword.
- A blue arrow points from the `while` block to the `while` keyword.
- A blue arrow points from the `while` block to the `while` keyword.

Output

```
Enter a positive number: 1  
Enter a positive number: 2  
Enter a positive number: 3  
Enter a positive number: 6  
Enter a positive number: 4  
Enter a positive number: 9  
Enter a positive number: 3  
Enter a positive number: 5  
Enter a positive number: -7
```

Handwritten notes:
- Blue circles around the input values 1, 2, and -7.

Output

```
Enter a positive number: -1
```

This example keeps asking the user to enter a positive number. The loop stops when the user enters 0 or a negative number:

CHOOSE THE CORRECT ANSWER

What is the main difference between a do/while loop and a while loop?

- A. ~~The do/while loop checks the condition first~~
- B. The do/while loop executes the code block at least once before checking the condition
- C. ~~The do/while loop only runs if the condition is initially false~~
- D. ~~The do/while loop is only used for infinite loops~~

What will the following code output?

```
int i = 5;  
do {  
    printf("%d\n", i);  
    i++;  
}
```

```
while (i < 5);
```

- A. 5
- B. No output
- C. 0
- D. 5 6 7 8 9

Which of the following correctly shows the syntax of a do/while loop?

- A. do { // code block } while (condition);
- B. ~~do while (condition) { // code block }~~
- C. ~~while (condition) do { // code block }~~
- D. ~~do { // code block } while condition;~~

Use the do/while loop to print i as long as i is less than 6.

```
int i = 1;  
{  
    printf("%d\n", i);  
    ;  
}  
(i < 6);
```

FOR LOOP

- When you **know exactly how many times** you want to loop through a block of code, use the for loop instead of a while loop:

```
#include <stdio.h>
```

```
int main() {
```

```
    int i;
```

initialization

condition

Increment or
decrement

```
    for (i = 0; i < 5; i++) {
```

```
        printf("%d\n", i);
```

```
    }
```

```
    return 0;
```

```
}
```

for i in range(5):
 print(i)

for i list:

PRINTS EVEN VALUES BETWEEN 0 AND 10:

```
#include <stdio.h>
```

```
int main() {
```

```
    int i;
```

```
    for (i = 0; i <= 10; i=i+2) {
```

```
        printf("%d\n", i);
```

```
    }
```

```
    return 0;
```

```
}
```

$0 = 0 + 2$
0
2
4

```
#include <stdio.h>
```

```
int main() {
```

```
    int i;
```

```
    for (i = 0; i <= 10; i++) {
```

```
        if (i%2==0) {
```

```
            printf("%d\n", i);
```

```
        }
```

```
    }
```

```
    return 0;
```

```
}
```

Handwritten notes and diagrams:
- A red circle around the loop condition $i \leq 10$.
- An arrow pointing from the circle to the value 10 in the output list.
- The text "2, 5 x" with an arrow pointing to the value 5 in the output list.
- The text "i%2==0" with an arrow pointing to the if statement.

Output

0

2

4

6

8

10

CALCULATES THE SUM OF NUMBERS FROM 1 TO 5:

```
#include <stdio.h>
```

```
int main() {
```

```
    int sum = 0; →
```

```
    int i;
```

```
    for (i = 1; i <= 5; i++) {
```

```
        sum = sum + i;
```

```
    }
```

```
    printf("Sum is %d", sum);
```

```
    return 0;
```

```
}
```

Handwritten calculations showing the iterative summing process:

$$\begin{aligned} 0 &= 0 + 1 = 1 \\ 1 &= 1 + 2 = 3 \\ 3 &= 3 + 3 = 6 \\ 6 &= 6 + 4 = 10 \\ 10 &= 10 + 5 = 15 \end{aligned}$$

Final result: $\text{sum} = 15$

Handwritten list of numbers and their sum:

$$\begin{array}{c} 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{array} = 15$$

PRINTS A COUNTDOWN FROM 5 TO 1:

```
#include <stdio.h>
```

```
int main() {
```

```
    int i;
```

```
    for (i = 5; i > 0; i--) {
```

```
        printf("%d\n", i);
```

```
    }
```

```
    return 0;
```

```
}
```

5
4
3
2
1

NESTED LOOPS

- It is also possible to place a loop inside another loop. This is called a **nested loop**.
- The "inner loop" will be executed one time for each iteration of the "outer loop":

```
#include <stdio.h>

int main() {
    int i, j;

    // Outer loop
    for (i = 1; i <= 2; ++i) {
        printf("Outer: %d\n", i); // Executes 2 times

        // Inner loop
        for (j = 1; j <= 3; ++j) {
            printf(" Inner: %d\n", j); // Executes 6 times (2 * 3)
        }
    }
    return 0;
}
```

Output

```
Outer: 1
 Inner: 1
 Inner: 2
 Inner: 3
Outer: 2
 Inner: 1
 Inner: 2
 Inner: 3
```


PRINT A SIMPLE MULTIPLICATION TABLE (1 TO 100):

```
#include <stdio.h>
```

```
int main() {  
    int i, j;  
  
    for (i = 1; i <= 10; i++) {  
        for (j = 1; j <= 10; j++) {  
            printf("%d * %d = %d  ", i, j, i * j);  
        }  
        printf("\n");  
    }  
  
    return 0;  
}
```

```
1 * 1 = 1  1 * 2 = 2  1 * 3 = 3  1 * 4 = 4  1 * 5 = 5  1 * 6 = 6  1 * 7 = 7  1 * 8 = 8  1 * 9 = 9  1 * 10 = 10  
2 * 1 = 2  2 * 2 = 4  2 * 3 = 6  2 * 4 = 8  2 * 5 = 10 2 * 6 = 12 2 * 7 = 14 2 * 8 = 16 2 * 9 = 18 2 * 10 = 20  
3 * 1 = 3  3 * 2 = 6  3 * 3 = 9  3 * 4 = 12 3 * 5 = 15 3 * 6 = 18 3 * 7 = 21 3 * 8 = 24 3 * 9 = 27 3 * 10 = 30  
4 * 1 = 4  4 * 2 = 8  4 * 3 = 12 4 * 4 = 16 4 * 5 = 20 4 * 6 = 24 4 * 7 = 28 4 * 8 = 32 4 * 9 = 36 4 * 10 = 40  
5 * 1 = 5  5 * 2 = 10 5 * 3 = 15 5 * 4 = 20 5 * 5 = 25 5 * 6 = 30 5 * 7 = 35 5 * 8 = 40 5 * 9 = 45 5 * 10 = 50  
6 * 1 = 6  6 * 2 = 12 6 * 3 = 18 6 * 4 = 24 6 * 5 = 30 6 * 6 = 36 6 * 7 = 42 6 * 8 = 48 6 * 9 = 54 6 * 10 = 60  
7 * 1 = 7  7 * 2 = 14 7 * 3 = 21 7 * 4 = 28 7 * 5 = 35 7 * 6 = 42 7 * 7 = 49 7 * 8 = 56 7 * 9 = 63 7 * 10 = 70  
8 * 1 = 8  8 * 2 = 16 8 * 3 = 24 8 * 4 = 32 8 * 5 = 40 8 * 6 = 48 8 * 7 = 56 8 * 8 = 64 8 * 9 = 72 8 * 10 = 80  
9 * 1 = 9  9 * 2 = 18 9 * 3 = 27 9 * 4 = 36 9 * 5 = 45 9 * 6 = 54 9 * 7 = 63 9 * 8 = 72 9 * 9 = 81 9 * 10 = 90  
10 * 1 = 10 10 * 2 = 20 10 * 3 = 30 10 * 4 = 40 10 * 5 = 50 10 * 6 = 60 10 * 7 = 70 10 * 8 = 80 10 * 9 = 90 10 * 10 = 100
```


Exercise?

What is the main use of a for loop in C?

- A. ~~To execute a block of code indefinitely~~
- B. To loop through a block of code a specific number of times
- C. ~~To check conditions without executing any code~~
- D. ~~To run a loop based on user input only~~

Use a for loop to print "Yes" 5 times:

```
for (int i = 0; i < 5; i++) {  
    printf("Yes\n");  
}
```

What is a nested loop?

- A. A loop that repeats indefinitely
- B. A loop placed inside another loop
- C. A loop with multiple conditions
- D. A loop that runs only once

How many times will the inner loop execute in the following code?

```
for (i = 1; i <= 2; ++i)  
{  
    for (j = 1; j <= 3; ++j) {  
        // code block  
    }  
}
```

- A. 3 times → 1
- B. 2 times → 6
- C. 5 times
- D. 6 times

CHECK WHETHER A NUMBER IS PRIME OR NOT

- **Write a program in C to check whether a number is prime or not.**

Idea

- Start checking from **2** up to **num - 1**.
- If the number is divisible by any value → **not prime**.
- If no divisor is found → **prime number**.

DO AT HOME

```
#include <stdio.h>

int main() {
    int num, i;
    printf("Enter a number: ");
    scanf("%d", &num);
    if (num < 2) {
        printf("%d is NOT a prime number\n", num);
        return 0; }
    for(i = 2; i < num; i++) {
        if(num % i == 0) {
            printf("%d is NOT a prime number\n", num);
            return 0; }
        }
    printf("%d is a prime number\n", num);
    return 0;
}
```



Happy Coding!